

# Tidy StatQuest - 01

Jonathan Kitt

2026-07-23



A few years ago, I started watching [Joshua Starmer's Youtube videos](#).

When his first book came out, I quickly got my hands on a copy, and I'm now the happy owner of his [StatQuest trilogy](#).

Josh's videos and books have been a huge help as I embarked on my statistics/data science journey.

For each chapter of his books, Josh wrote companion coding tutorials, available on Github.

In this series, I'll be “translating” the coding tutorials into {tidyverse} syntax.

Let's get started!

| Book                                          | Chapter                                    | Coding tutorial                                                                                                             |
|-----------------------------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| The StatQuest Illustrated Guide to Statistics | 01 - Fundamental Concepts in Statistics!!! | <a href="https://github.com/StatQuest/sigs/tree/main/chapter_01">https://github.com/StatQuest/sigs/tree/main/chapter_01</a> |

## Set up

We'll be using four packages:

|                          |                            |
|--------------------------|----------------------------|
| <code>{tidyverse}</code> | import and manipulate data |
| <code>{janitor}</code>   | clean data                 |
| <code>{rstatix}</code>   | calculate statistics       |
| <code>{gt}</code>        | generate nice tables       |

```
# Load packages
library(tidyverse)
library(janitor)
library(rstatix)
library(gt)
```

## Load data from a file

The data is available on the Github repository. We download it using the {readr} package:

```
file_url <- "https://raw.githubusercontent.com/StatQuest/sigs/refs/heads/main/chapter_01/spend_n_save"

spend_n_save <- read_tsv(file_url) |>
  # clean_names() default settings transform column names using snake case
  clean_names() |>
  # the 'id' variable is categorical, we transform it into a factor
  mutate(id = factor(id))
```

| id | num_apples |
|----|------------|
| 1  | 27         |
| 2  | 17         |
| 3  | 22         |
| 4  | 23         |
| 5  | 22         |

  

| id   | num_apples |
|------|------------|
| 5119 | 23         |
| 5120 | 28         |
| 5121 | 11         |
| 5122 | 27         |
| 5123 | 24         |

Print out the first and last rows:

```
# Print out the first 5 rows
head(spend_n_save, n = 5) |> gt()
```

```
# Print out the last 5 rows
tail(spend_n_save, n = 5) |> gt()
```

## Population Mean and Standard Deviation

---

Calculate the mean and standard deviation of the number of apples, using the `{rstatix}` package.

This function calculates different statistics for numerical variables. Since we transformed `id` into a factor, the code returns mean and standard deviation for `num_apples`. We use `mutate()` to round off the results.

```
pop_stats <- spend_n_save |>
  get_summary_stats(type = "mean_sd") |>
  mutate(across(where(is.double), ~ round(., 1)))

pop_stats |> gt()
```

| variable   | n    | mean | sd |
|------------|------|------|----|
| num_apples | 5123 | 19.9 | 5  |

---

| variable   | n | mean | sd  |
|------------|---|------|-----|
| num_apples | 5 | 21.2 | 2.8 |

## Estimated Mean and Standard Deviation

We set the seed for random number generation using `set.seed(42)`.

The `slice_sample()` function randomly picks `n` rows in a dataframe.

We then apply `get_summary_stats()` to the subset.

```
set.seed(42)

est_stats <- spend_n_save |>
  slice_sample(n = 5) |>
  get_summary_stats(type = "mean_sd") |>
  mutate(across(where(is.double), ~ round(., 1)))

est_stats |> gt()
```

To calculate the biased standard deviation, we use the `est_stats` table we just created, and then:

- we rename the `sd` variable to `sd_unbiased` using the `rename()` function
- we calculate the biased standard deviation using the following formula:

$$biased\ sd = \sqrt{(unbiased\ sd)^2 \times \frac{n-1}{n}}$$

- we round off the results

```
est_stats |>
  rename(sd_unbiased = sd) |>
  mutate(sd_biased = sqrt(sd_unbiased^2 * (n - 1) / n)) |>
  mutate(across(where(is.double), ~ round(., 1))) |>
  gt()
```

| variable   | n | mean | sd_unbiased | sd_biased |
|------------|---|------|-------------|-----------|
| num_apples | 5 | 21.2 | 2.8         | 2.5       |

## Simulate biased and unbiased estimates

After setting the seed, we replicate 1,000 experiments consisting in the following steps:

1. Draw 5 random rows from the table
2. Calculate the variance for `num_apples`
3. Store the 1,000 computed unbiased variances

We use the `replicate()` function, which takes two arguments:

- `n` for the number of replicates
- `expr` for the code describing the experiment

```
set.seed(42)

var_unbiased <- replicate(
  n = 1000,
  expr = {
    spend_n_save |>
      slice_sample(n = 5) |>
      summarise(var_unbiased = var(num_apples)) |>
      pull()
  }
)
```

Display the first 10 unbiased variances:

```
head(var_unbiased, 10)
```

```
[1]  7.7 21.3  4.0 10.7 16.3 12.8 17.7 13.8  6.3 55.3
```

We then create a table to summarise the results:

- `n = 5` (we want to use `n` rather than hard-code the values when computing the biased statistics)

| mean_sd_unbiased | mean_sd_biased |
|------------------|----------------|
| 4.8              | 4.3            |

- a column containing the 1,000 unbiased variances

Using `mutate()`, we then:

- calculate the unbiased standard deviation using the square-root of the unbiased variance
- calculate the biased variance using the formula above
- calculate the biased standard deviation using the square-root of the biased variance

Finally, we use `summarise()` to calculate the means of the biased and unbiased standard deviations from the 1,000 experiments.

```
tibble(
  n = 5,
  var_unbiased = var_unbiased
) |>
mutate(
  sd_unbiased = sqrt(var_unbiased),
  var_biased = var_unbiased * (n - 1) / n,
  sd_biased = sqrt(var_biased)
) |>
summarise(
  mean_sd_unbiased = round(mean(sd_unbiased), 1),
  mean_sd_biased = round(mean(sd_biased), 1)
) |>
gt()
```